



UNIVERSIDADE PAULISTA – UNIP

Av. Alberto Benassi, 200 - Parque das Laranjeiras - Araraquara - SP

CEP 14804-300 - Tel.: (16) 3336-1800

CIÊNCIA DA COMPUTAÇÃO

NOME DA ATIVIDADE APS – LINGUAGEM DE PROGRAMAÇÃO ORIENTADA A OBJETOS Educação Ambiental e a Vida em uma Grande Metrópole
Nome do aluno Gabriel Carrascosa
RA: G744CE-2

PROF. GUSTAVO AVELLAR

ARARAQUARA - SÃO PAULO

2023

APS – LINGUAGEM DE PROGRAMAÇÃO ORIENTADA A OBJETOS
Educação Ambiental e a Vida em uma Grande Metrópole

Atividades Práticas Supervisionadas para a obtenção de
nota para a graduação em Ciência da Computação
apresentado à Universidade Paulista – UNIP.

Orientador: Gustavo Avellar

ARARAQUARA - SÃO PAULO

2023

Biblioteca da Universidade Paulista - UNIP.

Guia de normalização para apresentação de trabalhos acadêmicos da
Universidade Paulista: ABNT / Biblioteca da Universidade Paulista - UNIP.
– 2021.

52 p. : il. color.

1. Normalização. 2. Trabalhos acadêmicos. 3. ABNT. I. Biblioteca da
Universidade Paulista – UNIP.

BANCA EXAMINADORA

_____/_____/_____

Prof. Gustavo Avellar

Universidade Paulista - UNIP

RESUMO

O projeto pode ser resumido na criação de um jogo simples utilizando a linguagem Java, com o tema de sustentabilidade e a vida em uma grande metrópole definido, onde se deve relacionar as questões de jogabilidade e se houver história ao tema proposto. Este jogo deve ser desenvolvido utilizando das boas práticas que o Java contém, questões como encapsulamento, polimorfismo, nomenclatura e de evitar repetição do código.

SUMÁRIO

1. OBJETIVOS DO TRABALHO.	6
1.1 Objetivos gerais.	6
1.2 Objetivos específicos.	6
2. INTRODUÇÃO.	7
2.1 Ideia do projeto.	7
2.2 Primeiros passos.	9
3. DESENVOLVIMENTO.	12
3.1 Aprimorando o projeto.	12
3.2 Contextualização da história do jogo.	16
4. APRESENTAÇÃO DO CÓDIGO.	22
4.1 Relatório do código.	22
5. DESAFIOS E OBSTÁCULOS.	72
5.1 Dificuldades enfrentadas.	72
5.2 Importância do trabalho para a formação.	73

1. OBJETIVO DO TRABALHO

1.1 Objetivos gerais

Entende-se como objetivo geral desenvolver um jogo eletrônico usando linguagens de programação ensinados no primeiro semestre de 2024, terceiro semestre do curso de Ciência da Computação em Linguagem de Programação Orientada a Objetos.

Pesquisar e coletar informações sobre as técnicas de desenvolvimento de jogos, seus conceitos, gêneros e ideias para criar uma estrutura e design adequados para o jogo, garantindo uma jogabilidade intuitiva e acessível para os usuários.

1.2 Objetivos específicos

O desenvolvimento de um código utilizando a linguagem aplicada em aula, o Java, tem como objetivo criar uma aplicação interativa, onde as ideias criativas devem ser estabelecidas pelo grupo responsável do projeto, de forma livre onde for necessário, mas que não fuja dos temas propostos, relacionados a sustentabilidade e uma vida em uma grande metrópole.

Para a realização desse projeto, é necessário que antes de tudo haja uma IDE (*Integrated Development Environment*), um Ambiente de Desenvolvimento Integrado, um programa de computador que seja capaz de prover ao programador uma interface simples e fácil de usar, que utilize de ferramentas uteis para proporcionar uma melhor experiencia de trabalho.

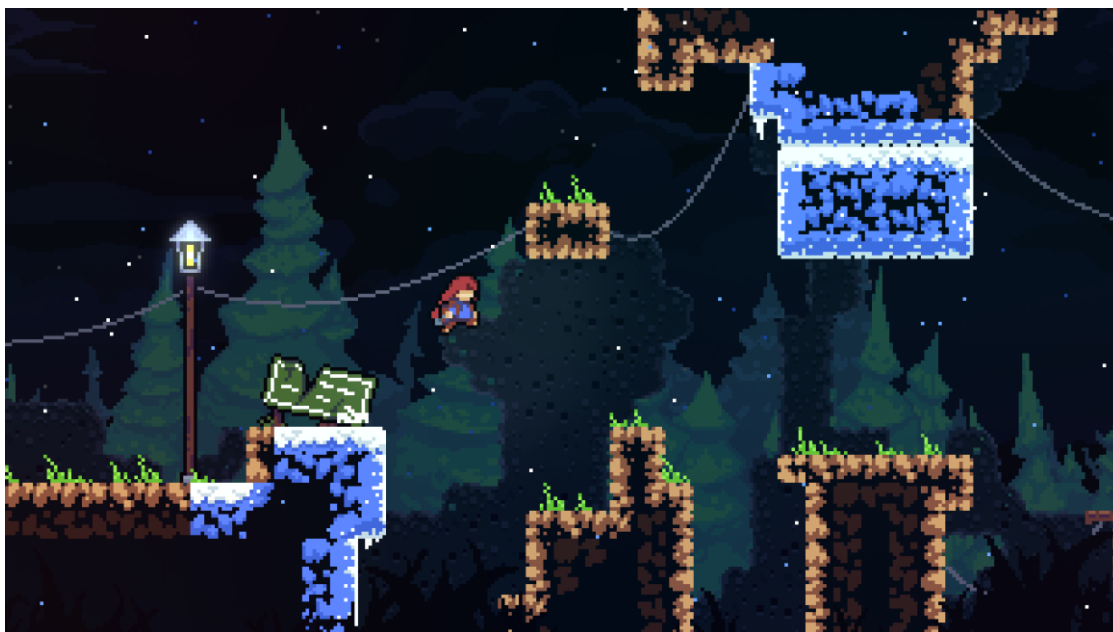
O escolhido para o desenvolvimento será o Eclipse, uma forte ferramenta quando a linguagem tratada é o Java, com diversos recursos que agilizam muito na hora de escrever certos códigos, como métodos “*Get*” e “*Set*” que são muito utilizados na linguagem.

2. INTRODUÇÃO

2.1 Ideia do projeto

As primeiras elaborações de ideias do que haveria dentro do jogo, jogabilidade, mecânicas e arte visual, foram inspiradas em um jogo de plataforma chamado “*Celeste*”, que é um estilo de jogo denominado “*2D*”, onde o jogador visualiza o personagem e os cenários de uma perspectiva lateral, oferecendo uma experiência imersiva e desafiadora. Esse estilo visual e mecânico, proporciona uma jogabilidade simples e intuitiva, onde o jogador deve guiar o personagem por diversos obstáculos e como o próprio nome diz, plataformas, mas mesmo com tamanha simplicidade, jogos deste gênero costumam exigir uma linha de aprendizagem muito interessante, onde resumindo, se trata de jogos fáceis de entender, mas difíceis de masterizar, por possuir diversas técnicas de jogabilidade que se misturam de uma maneira dinâmica e desafiadora com forme o decorrer do jogo.

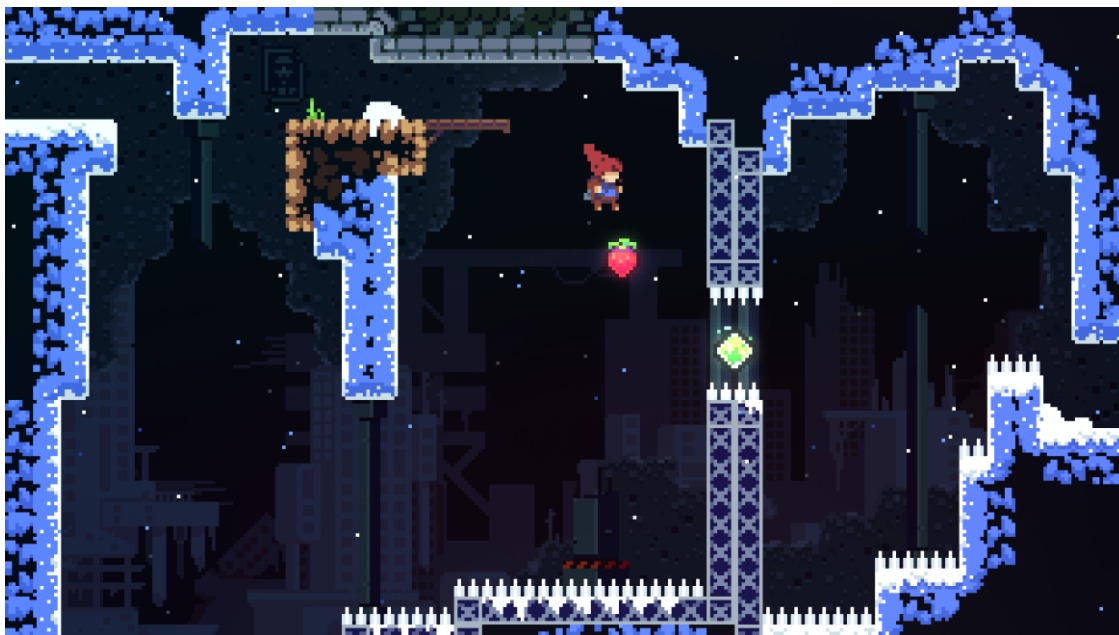
Figura 1 – Inspiração para começar o projeto.



Fonte: “*Celeste*”

A ideia inicial do projeto surgiu da admiração pelas mecânicas envolventes e a narrativa cativante de “*Celeste*”. Dentro desse jogo, há uma mecânica onde o jogador deve coletar morangos ao longo de sua jornada, essa ideia foi a principal motivação para o desenvolvimento, bastando apenas modificar os objetivos originais para que se adequa-se aos do projeto, o que significa que, neste projeto, o jogador deve coletar sacos de lixo ao longo de uma jornada e ao final será mostrado sua pontuação.

Figura 2 – Trocar a ideia de coletar morangos por sacos de lixo.



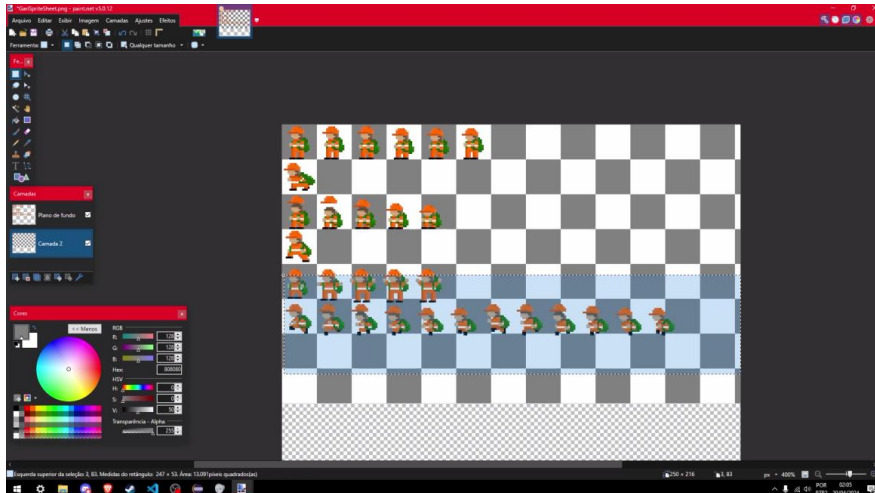
Fonte: “*Celeste*”

A partir dessa premissa, o desenvolvimento do jogo começou com a definição dos conceitos básicos, como a mecânica de movimento, salto e impulso, um sistema de escalada extremamente simples e claro, o sistema de coletar lixos ao colidir com eles. Diversas outras ideias foram adicionadas com o desenrolar do trabalho, como lugares onde o personagem passa e salva sua localização ao morrer, que ocorre ao colidir com espinhos. E foram adicionados trampolins e uma espécie de orbe que tem a funcionalidade de permitir que o jogador realize um impulso uma segunda vez, mesmo após já ter realizado uma vez no ar, que normalmente só reiniciaria ao encostar no chão.

2.2 Primeiros passos

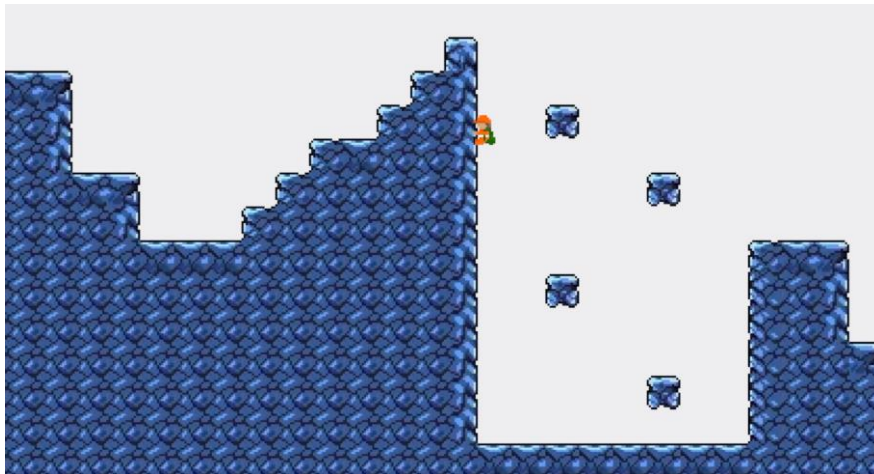
Serão abordados aspectos relacionados as primeiras coisas que foram desenvolvidas após o planejamento das ideias, como foi estabelecido que o jogo será de plataforma e conterá um sistema de escalada e de coletar lixos, o próximo passo foi desenvolver algum protótipo de jogabilidade, que já de começo, mesmo com alguns erros aqui ali, foi um sucesso nas questões de movimentação, pulo e escalada, contendo uma pequena fase bem simples, mas que já dava um bom direcionamento para onde o projeto se encaminharia.

Figura 3 – Personagem principal sendo desenhado.



Fonte: “Paint.net”

Figura 4 – Primeiro protótipo.



Fonte: “Eclipse IDE”

Mais tarde, foi necessário desenvolver as questões artísticas, a parte visual do jogo, que com ajuda de várias imagens da internet, se chegou em um resultado bem interessante, montanhas de terra com uma grama em cima, um céu azul bem simples, casas ao longo do mapa e claro, lixos para coletar.

O lixeiro, que é o nosso personagem principal, teria uma pequena história no começo do jogo, para explicar a situação na qual ele se meteu, que é a de ter que escalar uma grande montanha para pegar um suposto atalho, é uma história bem boba, mas que encaixa muito bem na narrativa do jogo.

Figura 5 – Parte visual já bem desenvolvida.



Fonte: “Eclipse IDE”

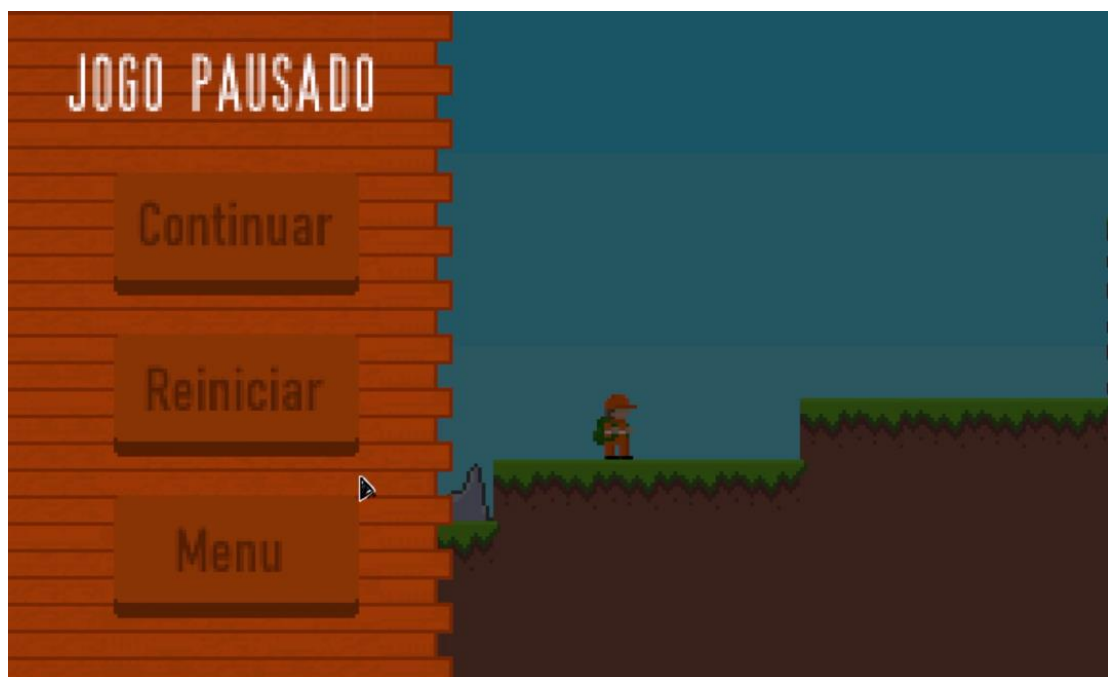
Documentar como foi o desenrolar do projeto é um pouco complicado, visto que várias partes do projeto foram sendo pensadas e desenvolvidas simultaneamente, hora se programava uma nova mecânica, hora voltava na parte artística e mudava a cor de uma parte do cenário, mas é possível organizar em uma sequência de: Planejamento de ideias; Busca por materiais visuais para auxiliar na parte gráfica do jogo; Busca por questões de jogabilidade; desenvolvimento de uma interface inicial, para que o jogo não começasse do nada, contendo botões para começar um novo jogo, continuar e sair; Botão para pausar o jogo e nela opções de reiniciar a fase, voltar ao menu e continuar jogando e uma tela final onde mostraria a pontuação do jogador.

Figura 6 – Menu e mecânica de impulso já bem desenvolvidos.



Fonte: "Eclipse IDE"

Figura 7 – Tela de "*Pause*" desenvolvida.



Fonte: “Eclipse IDE”

3. DESENVOLVIMENTO

3.1 Aprimorando o projeto

Quando o projeto foi inicializado, a ideia de construção dos mapas seria de tentar desenvolver algo próximo do jogo “*Celeste*”, mas de início não parecia uma ideia simples, por conta do estilo de fases que o jogo original apresenta, onde ao encostar no final de uma fase a câmera desliza para o começo da fase seguinte, talvez até desse para desenvolver algo parecido, mas logo foi descartada, no lugar, foi feito um sistema de câmera que apenas acompanha o jogador por todo o mapa, mais voltado pro jogo “*Super Mario*”, mas com um mapa bem mais extenso, tanto na horizontal quanto vertical.

Um dos desafios enfrentados, foi a escolha dessa câmera, que reduzia bruscamente a performance, pois o algoritmo responsável por desenhar o mapa funcionava de uma forma que, no começo do jogo, se colocava todos os objetos, seja casas, lixos, espinhos ou os blocos de montanha de uma só vez, o que causava uma queda enorme na taxa de quadros, já que o Java tinha que desenhar todas as entidades, mesmo que elas estivessem fora da tela, tornando o jogo bem travado, então a solução foi desenvolver um sistema que desenhava apenas os objetos que fossem visíveis para o jogador, o que tornou o jogo com um melhor desempenho.

E foi nesta etapa do projeto, onde já se tinha uma parte do jogo desenvolvida, um sistema minimamente aceitável de performance, um mapa legal com alguns lixos aqui e ali, umas casinhas e outros detalhes, que o trabalho ficou muito tempo estagnado, pela falta de ideias e pelo cansaço de ter realizado tanta programação e ainda parecer que não tinha feito quase

nada, mas após dias sem mexer no projeto, ele ainda precisava ser finalizado, sendo necessário criar novas áreas para o jogador explorar, colocar mais lixos espalhados pelo mapa e volta em meia esconder alguns segredos no jogo.

E restando alguns dias para a apresentação, foi quando o jogo deu seu maior salto em termos de detalhes e áreas para se explorar, um pouco antes desse momento, o jogo havia uma fase inicial de grama, uma caverna de terra que ao final, se conectava com uma nova caverna de pedra, mas que ainda não tinha praticamente nada dentro dela, duas ou três casinhas escondidas aos arredores do mapa, alguns trampolins, locais para salvar a posição do jogador e vários espinhos.

Mas após as atualizações, o jogo já continha um final para a caverna de pedra, um orbe onde se reiniciava o impulso quando o personagem colidia com ele, uma vez tendo realizado o impulso e não encostado no chão, uma fase final com alguns prédios, uma cena de introdução e uma de finalização com a pontuação feita pelo jogador, contendo a quantidade de lixos coletados, mortes e tentativas realizadas e alguns novos detalhes, como graminhas e pedrinhas.

Figura 8 – Final da caverna de pedra.



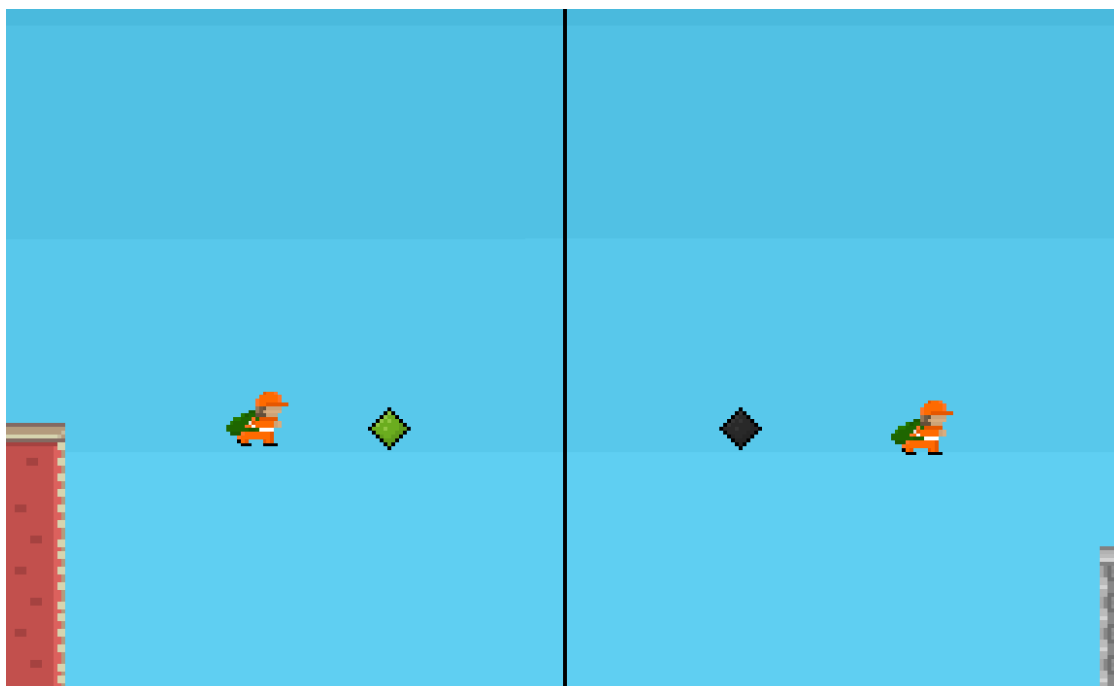
Fonte: "Eclipse IDE"

Figura 9 – Área final do jogo, com prédios escaláveis.



Fonte: “Eclipse IDE”

Figura 10 – Orbe que “reseta” o impulso do lixeiro.



Fonte: “Eclipse IDE”

Figura 11 Cena de introdução com diálogos.



Fonte: "Eclipse IDE"

Figura 12 – Cena final mostrando a pontuação do jogador.



Fonte: "Eclipse IDE"

3.2 Contextualização da história do jogo

O jogo sendo apenas um menu que ao apertar no botão de iniciar novo jogo, levava o jogador ao mapa, não tinha nenhum contexto ou explicação de como ele veio parar lá, nada, portanto o jogo precisava de um mínimo de enredo, por mais simples que fosse, e foi desenvolvido em conjunto com o resto das mecânicas, arte visual e tudo mais.

A história começa com nosso protagonista, um gari, começando mais um dia de trabalho, mas naquele dia, ele resolveu reclamar de sua fatídica vida, com um tom irônico, dizendo que realmente adora o trabalho dele, mas logo o outro lixeiro, que estava dirigindo o caminhão de lixo o ouve e decide o abandonar, fazendo com que ele tenha que caminhar a pé até chegar a ele novamente, mas o nosso herói não se dá por vencido, e decide escalar uma montanha imensa, dizendo conhecer um enorme atalho.

Logo adiante, o jogo nos apresenta os botões de movimentação e pulo, para que o jogador possa saber o que fazer a seguir.

Figura 13 – Começo do jogo.



Fonte: “Eclipse IDE”

Mais adiante, após alguns montes e barrancos, o personagem passa pela primeira casa com lixo coletável, apresentando ao jogador a mecânica básica de coleta, que é realizada simplesmente ao colidir com o saco de lixo.

Figura 14 – Primeiro saco de lixo do jogo.



Fonte: “Eclipse IDE”

E logo na sequência, é apresentado o sistema de “*Checkpoint*”, em que ao colidir com espinhos, o jogador é redirecionado ao último ponto salvo.

Figura 15 – Sistema de “*Checkpoint*”.



Fonte: “Eclipse IDE”

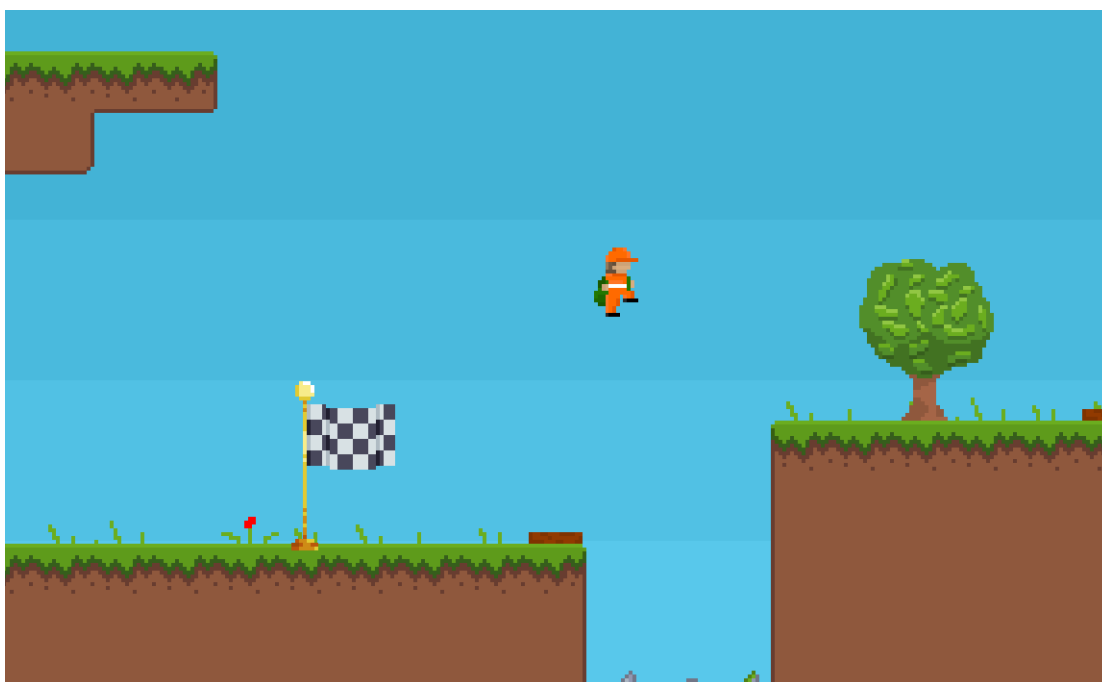
Uma caverna de terra é a próxima área do jogo, onde se encontra uma barraquinha com algum cidadão vivendo nela e um saco de lixo na frente.

Figura 16 – Caverna de terra.



Fonte: “Eclipse IDE”

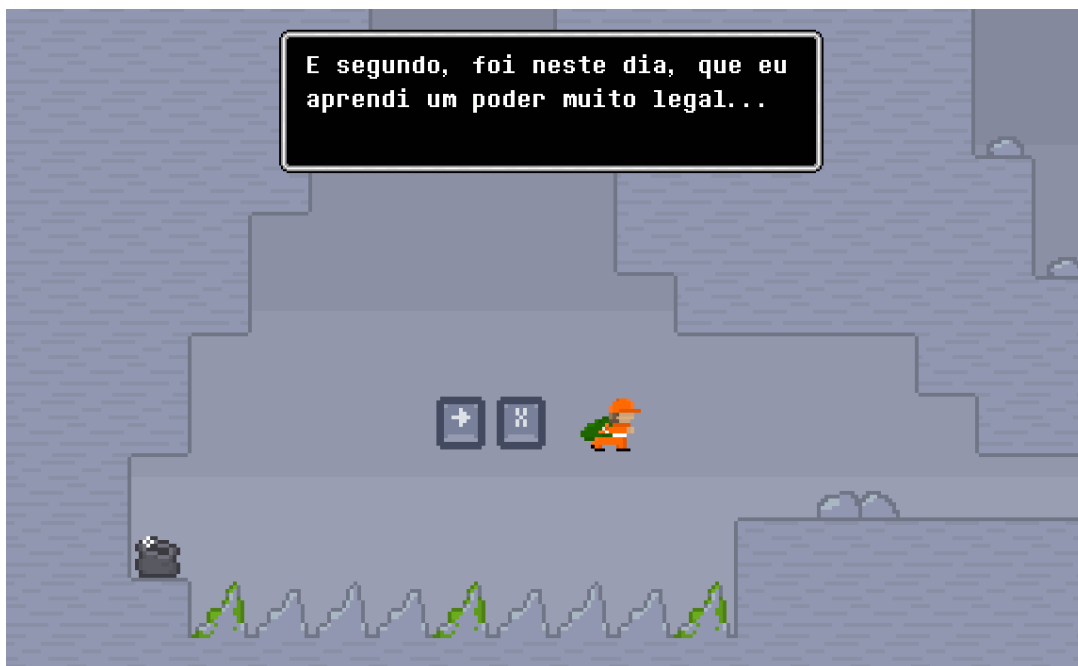
Figura 17 – Introdução ao trampolim.



Fonte: “Eclipse IDE”

Uma caverna de pedra então será a nova área, onde se encontra um buraco onde o jogador acaba caindo nele e descobrindo um novo poder, uma nova habilidade, o impulso.

Figura 18 – Caverna de pedra e introdução à mecânica de impulso.



Fonte: “Eclipse IDE”

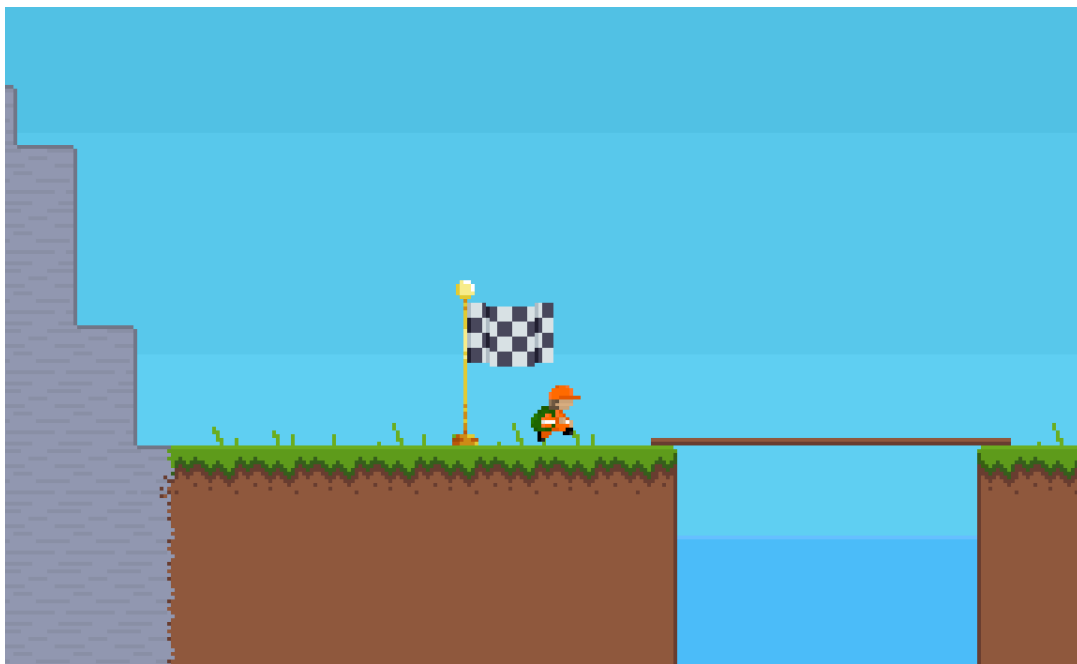
Figura 19 – Apresentação ao orbe de “Reset” de impulso.



Fonte: “Eclipse IDE”

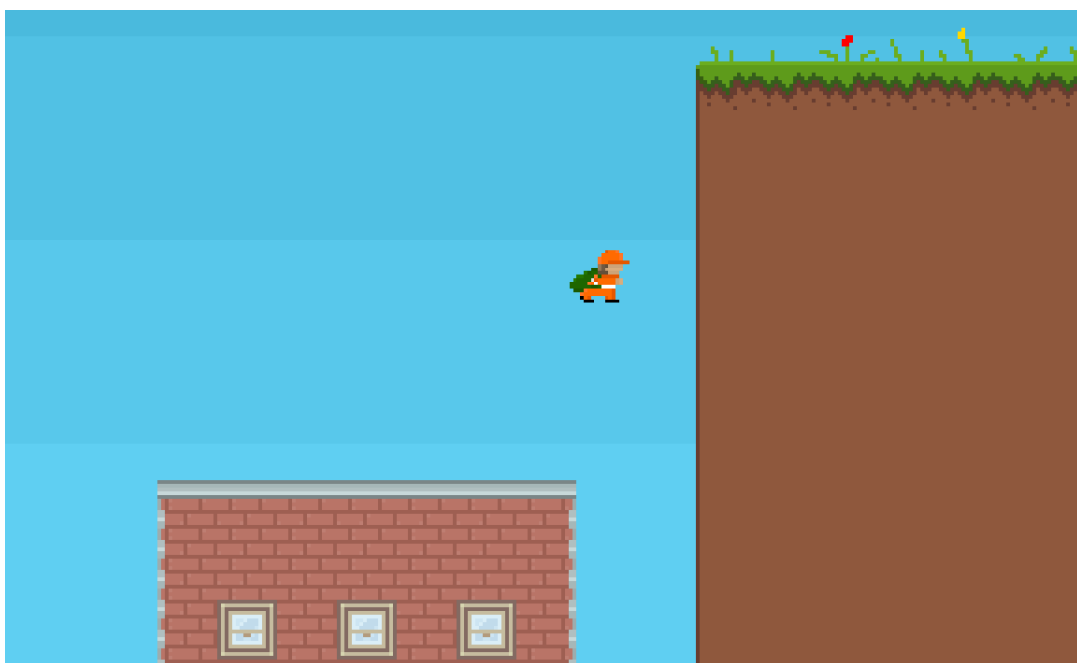
Ao sair da caverna de pedra, temos a parte final do jogo, onde temos uma pequena ponte e alguns prédios escaláveis para chegar ao topo de uma montanha enorme, com uma armadilha no final.

Figura 20 – Área final do jogo.



Fonte: “Eclipse IDE”

Figura 21 – Montanha final.



Fonte: “Eclipse IDE”

Figura 22 – Armadilha cômica.



Fonte: “Eclipse IDE”

E ao final da jornada, nosso herói enfim retorna à casa.

Figura 23 – A volta dos que não foram.



Fonte: “Eclipse IDE”

4. APRESENTAÇÃO DO CÓDIGO

4.1 Relatório do código

A seguir será apresentado os códigos do projeto, mostrando apenas as classes mais importantes, como a que dá vida ao programa, a do jogador, da fase, dos “*Inputs*” de teclado e “*mouse*”, superclasses e algumas outras:

Classe Main:

```
package main;

public class Main

{

    public static void main(String[] args)

    {

        new Game();

    }

}
```

Classe Game:

```
package main;
```

```
import java.awt.Graphics;

import entities.CheckPoint;

import gamestates.CutScene;

import gamestates.End;

import gamestates.Gamestate;

import gamestates.Menu;

import gamestates.Playing;

import gamestates.Introduction;
```

```
public class Game implements Runnable

{

    //Dentro de Game chamamos todas as funções do jogo:

    //A janela, que corresponde como uma borda para uma pintura

    //O panel, que corresponde à pintura em si

    private GameWindow gameWindow;

    private GamePanel gamePanel;

    private Thread gameThread;
```

```
//Constante de quantas atualizações de repaint()
```

```
public final static int UPS_SET = 5;
```

```
public static int fps = 1000;
```

```
public static int ups = UPS_SET;
```

```
    //Algumas variáveis sobre pontuação do player
```

```
public static int collectedTrashes=0;
```

```
public static int deathCount=0;
```

```
public static int attemptCount=0;
```

```
public static float time;
```

```
double timePerFrame = 1000000000.0 / fps;
```

```
double timePerUpdate = 1000000000.0 / ups;
```

```
private Introduction introduction;
```

```
private Menu menu;
```

```
private Playing playing;
```

```
private CutScene cutscene;
```

```
private End end;
```

////Variáveis para definir o tamanho da tela e dos sprites

```
public final static float SCALE = 4f;
```

```
public final static int TILES_DEFAULT_SIZE = 16;
```

```
public final static int TILES_IN_WIDTH = 18;
```

```
public final static int TILES_IN_HEIGHT = 11;
```

```
public final static int TILES_SIZE = (int) (TILES_DEFAULT_SIZE *  
    SCALE);
```

```
public final static int GAME_WIDTH = TILES_SIZE *  
    TILES_IN_WIDTH;
```

```
public final static int GAME_HEIGHT = TILES_SIZE *  
    TILES_IN_HEIGHT;
```

```
public static boolean potatoMode = false;
```

```
public static int slowMotion = UPS_SET;
```

```
public Game()
```

```
{  
  
    initClasses();  
  
    gamePanel = new GamePanel(this);  
  
    gameWindow = new GameWindow(gamePanel);  
  
    gamePanel.requestFocus();  
  
    //Inicialização do Loop do Jogo  
  
    gameThread = new Thread(this);  
  
    gameThread.start();  
  
}
```

```
private void initClasses()  
  
{  
  
    introduction = new Introduction(this);
```

```
menu = new Menu(this);

playing = new Playing(this);

cutscene = new CutScene(this);

end = new End(this);

}
```

```
public void update()

{

timePerFrame = 1000000000.0 / fps;

timePerUpdate = 1000000000.0 / ups;
```

```
//Forma de trocar entre o menu e as fases
```

```
switch(Gamestate.state)
```

```
{
```

```
case INTRODUCTION:
```

```
{
```

```
introduction.update();
```

```
break;
```

```
}
```

```
case MENU:
```

```
{  
  
    menu.update();  
  
    break;  
  
}  
  
case PLAYING:  
  
    {  
  
        playing.update();  
  
        break;  
  
    }  
  
case CUTSCENE:  
  
    {  
  
        cutscene.update();  
  
        CheckPoint.restartAllCheckpoints();  
  
        break;  
  
    }  
  
case END:  
  
    {  
  
        end.update();  
  
        break;  
  
    }
```

```
case QUIT:
```

```
default:
```

```
System.exit(0);
```

```
break;
```

```
}
```

```
}
```

```
//Método que desenha na tela a fase atual, como o menu ou mesmo uma  
fase do jogo
```

```
public void render(Graphics g)
```

```
{
```

```
switch(Gamestate.state)
```

```
{
```

```
case INTRODUCTION:
```

```
{
```

```
introduction.draw(g);
```

```
break;
```

```
}
```

```
case MENU:
```

```
{
```

```
menu.draw(g);
```



```
break;

}

case PLAYING:

{

playing.draw(g);

break;

}

case CUTSCENE:

{

cutscene.draw(g);

break;

}

case END:

{

end.draw(g);

break;

}

default:

break;

}
```

```
}
```

```
@Override
```

```
public void run()
```

```
{
```

```
    long previousTime = System.nanoTime();
```

```
    int frames = 0;
```

```
    int updates = 0;
```

```
    long lastCheck = System.currentTimeMillis();
```

```
    double deltaU = 0;
```

```
    double deltaF = 0;
```

```
    while (true) {
```

```
        long currentTime = System.nanoTime();
```

```
        deltaU += (currentTime - previousTime) / timePerUpdate;
```

```
        deltaF += (currentTime - previousTime) / timePerFrame;
```

```
        previousTime = currentTime;
```

```
if (deltaU >= 1) {  
  
    update();  
  
    updates++;  
  
    deltaU--;  
  
}
```

```
if (deltaF >= 1) {  
  
    gamePanel.repaint();  
  
    frames++;  
  
    deltaF--;  
  
}
```

```
if (System.currentTimeMillis() - lastCheck >= 1000) {  
  
    lastCheck = System.currentTimeMillis();  
  
    //System.out.println("FPS: " + frames + " | UPS: " +  
        updates);  
  
    frames = 0;  
  
    updates = 0;  
  
}  
  
}
```

```
}
```

```
public void windowFocusLost()
```

```
{
```

```
System.out.println("Window Focus Lost");
```

```
playing.getPlayer().resetDirectionBooleans();
```

```
menu.getPlayer().resetDirectionBooleans();
```

```
}
```

```
public Introduction getIntroduction()
```

```
{
```

```
return introduction;
```

```
}
```

```
public Menu getMenu()
```

```
{
```

```
return menu;
```

```
}
```

```
public Playing getPlaying()
```

```
{  
  
    return playing;  
  
}
```

```
    public CutScene getCutscene()  
  
    {  
  
        return cutscene;  
  
    }
```

```
    public End getEnd()  
  
    {  
  
        return end;  
  
    }  
  
}
```

Classe GameWindow:

```
package main;
```

```
import java.awt.event.WindowEvent;
```

```
import java.awt.event.WindowFocusListener;

import java.awt.image.BufferedImage;

import javax.swing.JFrame;

import utiliz.LoadSave;

public class GameWindow

{

    private JFrame jframe;


    public GameWindow(GamePanel gamePanel)

    {

        jframe = new JFrame();


        //Configurações de iniciar a tela no meio do monitor, fechar
        //programa automático e tornar a tela visível


        //Dependendo da ordem em que os itens abaixo são
        //organizados, podem dar uma bugada em algumas funções no
        //jogo


        jframe.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);

        jframe.add(gamePanel);


        jframe.setResizable(false);
```

```
jframe.pack();
```

```
jframe.setLocationRelativeTo(null);
```

```
jframe.setVisible(true);
```

```
jframe.addWindowFocusListener
```

```
(new WindowFocusListener()
```

```
{
```

```
@Override
```

```
public void windowLostFocus(WindowEvent e)
```

```
{
```

```
gamePanel.getGame().windowFocusLost();
```

```
}
```

```
@Override
```

```
public void windowGainedFocus(WindowEvent e)
```

```
{
```

```
System.out.println("Window Gained Focus");
```

```
}
```

```
}
```

```
);
```

```
//Definir título e ícone da janela
```

```
BufferedImage icon =
```

```
    LoadSave.GetSpriteAtlas(LoadSave.ICON_ATLAS);
```

```
jframe.setTitle("APS");
```

```
jframe.setIconImage(icon);
```

```
}
```

```
}
```

Classe GamePanel:

```
package main;
```

```
import java.awt.Dimension;
```

```
import java.awt.Graphics;
```

```
import javax.swing.JPanel;
```

```
import inputs.KeyboardInputs;
```

```
import inputs.MouseInputs;
```

```
import static main.Game.GAME_HEIGHT;
```

```
import static main.Game.GAME_WIDTH;
```



```
public class GamePanel extends JPanel

{

    private MouseInputs mouseInputs;

    private Game game;


    public GamePanel(Game game)

    {

        setPanelSize();

        this.game = game;


        //Inputs de mouse e teclado

        mouseInputs = new MouseInputs(this);

        addKeyListener(new KeyboardInputs(this));

        addMouseListener(mouseInputs);

        addMouseMotionListener(mouseInputs);

    }


    //Definir o tamanho da tela dentro da janela

    private void setPanelSize()
```

```
{  
  
    Dimension size = new Dimension(GAME_WIDTH, GAME_HEIGHT);  
  
    setPreferredSize(size);  
  
  
    System.out.println("Size: " + GAME_WIDTH + " | " + GAME_HEIGHT);  
  
}
```

//Método para desenhar o visual do jogo, proporcionando a alteração de FPS,

//já que o tempo de atualização da lógica do jogo não será alterada

```
public void paintComponent(Graphics g)
```

```
{  
  
    super.paintComponent(g);  
  
    game.render(g);  
  
}
```

```
public Game getGame()
```

```
{  
  
    return game;  
  
}  
  
}
```

Classe KeyboardInputs:

```
package inputs;
```

```
import java.awt.event.KeyEvent;
```

```
import java.awt.event.KeyListener;
```

```
import gamestates.Gamestate;
```

```
import main.GamePanel;
```

```
public class KeyboardInputs implements KeyListener
```

```
{
```

```
    private GamePanel gamePanel;
```

```
    public KeyboardInputs(GamePanel gamePanel)
```

```
{
```

```
        this.gamePanel = gamePanel;
```

```
}
```

```
@Override
```

```
public void keyReleased(KeyEvent e)

{

switch(Gamestate.state)

{

case INTRODUCTION:

{

    gamePanel.getGame().getIntroduction().keyReleased(e);

break;

}

case MENU:

{

gamePanel.getGame().getMenu().keyReleased(e);

break;

}


case PLAYING:

{

                                gamePanel.getGame().getPlaying().keyReleased(e);

break;

}

case CUTSCENE:
```

```
{  
  
    gamePanel.getGame().getCutsScene().keyReleased(e);  
  
    break;  
  
}  
  
case END:  
  
    {  
  
        gamePanel.getGame().getEnd().keyReleased(e);  
  
        break;  
  
    }  
  
default:  
  
    break;  
  
}  
  
}
```

@Override

```
public void keyPressed(KeyEvent e)  
  
    {  
  
        switch(Gamestate.state)  
  
        {  
  
            case INTRODUCTION:
```

```
{

gamePanel.getGame().getIntroduction().keyPressed(e);

break;

}

case MENU:

{

gamePanel.getGame().getMenu().keyPressed(e);

break;

}

case PLAYING:

{

gamePanel.getGame().getPlaying().keyPressed(e);

break;

}

case CUTSCENE:

{

gamePanel.getGame().getCutscene().keyPressed(e);

break;

}

case END:
```

```
{  
  
gamePanel.getGame().getEnd().keyPressed(e);  
  
break;  
  
}  
  
default:  
  
break;  
  
}  
  
}
```

@Override

```
public void keyTyped(KeyEvent e) {}  
  
}
```

Classe MouseInputs:

```
package inputs;  
  
import java.awt.event.MouseEvent;  
  
import java.awt.event.MouseListener;  
  
import java.awt.event.MouseMotionListener;
```

```
import gamestates.Gamestate;
```

```
import main.GamePanel;
```

```
public class MouseInputs implements MouseListener, MouseMotionListener
```

```
{
```

```
    private GamePanel gamePanel;
```

```
    public MouseInputs(GamePanel gamePanel)
```

```
{
```

```
        this.gamePanel = gamePanel;
```

```
}
```

```
    //Movimentação comum do Mouse
```

```
    @Override
```

```
    public void mouseMoved(MouseEvent e)
```

```
{
```

```
    switch (Gamestate.state)
```

```
{
```

```
        case MENU:
```

```
{
```



```
gamePanel.getGame().getMenu().mouseMoved(e);

break;

}

case PLAYING:

{

gamePanel.getGame().getPlaying().mouseMoved(e);

break;

}

default:

break;

}

}

//Mouse Pressionado

@Override

public void mousePressed(MouseEvent e)

{

switch(Gamestate.state)

{

case INTRODUCTION:

{
```

```
        gamePanel.getGame().getIntroduction().mousePressed(e);

        break;

    }

    case MENU:

    {

        gamePanel.getGame().getMenu().mousePressed(e);

        break;

    }

    case PLAYING:

    {

        gamePanel.getGame().getPlaying().mousePressed(e);

        break;

    }

    default:

        break;

    }

}

//Soltar o Mouse

@Override

public void mouseReleased(MouseEvent e)
```

```
{  
  
switch(Gamestate.state)  
  
{  
  
case INTRODUCTION:  
  
{  
  
        gamePanel.getGame().getIntroduction().mouseReleased(e);  
  
break;  
  
}  
  
case MENU:  
  
{  
  
gamePanel.getGame().getMenu().mouseReleased(e);  
  
break;  
  
}  
  
case PLAYING:  
  
{  
  
gamePanel.getGame().getPlaying().mouseReleased(e);  
  
break;  
  
}  
  
default:  
  
break;
```

```
}
```

```
}
```

```
//Clique do Mouse
```

```
@Override
```

```
public void mouseClicked(MouseEvent e) {}
```

```
//Segurar e arrastar
```

```
@Override
```

```
public void mouseDragged(MouseEvent e) {}
```

```
@Override
```

```
public void mouseEntered(MouseEvent e) {}
```

```
@Override
```

```
public void mouseExited(MouseEvent e) {}
```

```
}
```

Classe Playing:

Esta classe é responsável pela criação e questões de atualizações de imagem e de atualizações da lógica do jogo, como a física da gravidade, pulo, tempo que leva para trocar uma animação do jogador para outra e entre muitas outras, e justamente por ser uma das classes mais importantes do projeto, seu tamanho se deu por volta de 900 linhas, o que tornaria esse documento extremamente longo, portanto, várias linhas de criação de entidades e dessas questões de atualizações tiveram que ser removidas, foi deixada apenas algumas linhas referentes ao saco de lixo, e como diversas outras entidades usam do mesmo princípio, apenas esta será mostrada neste relatório.

```
package gamestates;
```

```
import java.awt.Color;
```

```
import java.awt.Graphics;
```

```
import java.awt.event.KeyEvent;
```

```
import java.awt.event.MouseEvent;
```

```
import java.awt.image.BufferedImage;
```

```
import entities.CheckPoint;
```

```
import entities.InvisibleWall;
```

```
import entities.MiniPlayer;
```

```
import entities.OrbDash;
```

```
import entities.Player;
```

```
import entities.Spike;
```

```
import entities.Trampoline;
```

```
import entities.Trash;
```

```
import entities.TriggerEvent;
```

```
import entities.TriggerSkyChange;
```

```
import entities.Vehicle;
```

```
import levels.AnimatedProp;
```

```
import levels.LevelManager;
```

```
import levels.Props;
```

```
import main.Game;
```

```
import ui.PauseOverlay;
```

```
import ui.TextBox;
```

```
import utilz.LoadSave;
```

```
public class Playing extends State implements Statemethods

{

    public static int trashAnimationTick, trashAnimationIndex,
        trashAnimationSpeed = 16;


    private LevelManager levelManager;

    private Player player;

    private MiniPlayer miniplayer;

    private TriggerSkyChange triggerskychange;

    private AnimatedProp eye, moveKeys, upKeys;

    private InvisibleWall wallOne, wallTwo;

    private Vehicle garbageTruck;


    private PauseOverlay pauseOverlay;

    public static boolean paused = false, active = true;


    private int distance = (int) (212 * Game.SCALE);


    public static int garbageTruckSpeed=0;
```

```
private Trash objTrashOne, objTrashTwo, objTrashThree,  
            objTrashFour, objTrashFive, objTrashSix, objTrashSeven,  
  
objTrashEight, objTrashNine, objTrashTen;
```

```
public static int xLevelOffset, yLevelOffset;
```

```
private int leftBorder = (int)(0.46 * Game.GAME_WIDTH);
```

```
private int rightBorder = (int)(0.54 * Game.GAME_WIDTH);
```

```
private          int          levelTilesWide          =  
LoadSave.GetLevelData(LoadSave.LEVEL_ONE_DATA)[0].length;
```

```
private int maxTilesOffsetX = levelTilesWide - Game.TILES_IN_WIDTH;
```

```
private int maxLevelOffsetX = maxTilesOffsetX * Game.TILES_SIZE;
```

```
private int topBorder = (int)(0.4 * Game.GAME_HEIGHT);
```

```
private int bottomBorder = (int)(0.6 * Game.GAME_HEIGHT);
```

```
private          int          levelTilesHigh          =  
LoadSave.GetLevelData(LoadSave.LEVEL_ONE_DATA)[1].length;
```

```
private int maxTilesOffsetY = levelTilesHigh - Game.TILES_IN_WIDTH;
```

```
private int maxLevelOffsetY = maxTilesOffsetY * Game.TILES_SIZE;
```

```
private static int subTick=0, tick=0;
```

```
private static int slowMotionTickValue=Game.UPS_SET/Game.slowMotion,  
slowMotionTick=slowMotionTickValue;
```



```
private boolean textBoxTrigger = false;
```

```
//Player começa nesta coordenada
```

```
public static int playerInitialX=424;
```

```
public static int playerInitialY=2800;
```

```
//Céu com a cor padrão:
```

```
public static int red = brightSkyRed, blue = brightSkyBlue, green =  
brightSkyGreen, alpha = brightSkyAlpha;
```

```
private BufferedImage
```

```
backgroundImage =  
LoadSave.GetSpriteAtlas(LoadSave.PLAYING_BACKGROUND),
```

```
backgroundImageTwo =  
LoadSave.GetSpriteAtlas(LoadSave.PLAYING_BACKGROUND TWO),
```

```
backgroundImageThree =  
LoadSave.GetSpriteAtlas(LoadSave.PLAYING_BACKGROUND THREE),
```

```
backgroundImageFour =  
LoadSave.GetSpriteAtlas(LoadSave.PLAYING_BACKGROUND FOUR),
```

```
backgroundImageFive =  
LoadSave.GetSpriteAtlas(LoadSave.PLAYING_BACKGROUND FIVE),
```

```
houseGrass = LoadSave.GetSpriteAtlas("/HouseGrass.png"),
```

```
houseCommon = LoadSave.GetSpriteAtlas("/HouseCommon.png"),
```

```
houseBroken = LoadSave.GetSpriteAtlas("/HouseBroken.png"),
```

```
tent = LoadSave.GetSpriteAtlas("/Tent.png"),
```

```
private void initClasses()
```

```
{
```

```
levelManager = new LevelManager(game, LoadSave.LEVEL_ONE_DATA);
```

```
player = new Player(playerInitialX * Game.SCALE , playerInitialY *  
Game.SCALE, (int)(18 * Game.SCALE), (int)(18 * Game.SCALE));
```

```
player.loadLevelData(levelManager.getCurrentLevel().getLevelData());
```

```
miniplayer = new MiniPlayer(playerInitialX * Game.SCALE , playerInitialY *  
Game.SCALE, (int)(-100 * Game.SCALE), (int)(-100 * Game.SCALE));
```

```
miniplayer.loadLevelData(levelManager.getCurrentLevel().getLevelData());
```

```
triggerskychange = new TriggerSkyChange(player);
```

```
eye = new AnimatedProp("/Eye.png", 4204, 2664, 5, 24, 8, 100);
```

```
moveKeys = new AnimatedProp("/MoveKeys.png", 480, 2720-28, 32, 32, 2,  
100);
```

```
upKeys = new AnimatedProp("/UpKeys.png", 480+64, 2720-28, 16, 64, 4, 100);
```

```
//Parte da criação de dezenas de entidades
```

```
createTrash();
```

```
createCheckPoint();
```

```
createSpike();
```

```
createTrampoline();
```

```
createOrbdash();
```

```
createTriggerEvent();
```

```
pauseOverlay = new PauseOverlay(this);
```

```
}
```

```
public Playing(Game game)
```

```
{
```

```
super(game);
```

```
initClasses();
```

```
}
```

```
public void unpauseGame()
```

```
{
```

```
paused = false;
```

```
}
```

```
public void windowFocusLost()
```

```
{
```

```
player.resetDirectionBooleans();
```

```
}
```

```
public Player getPlayer()
```

```
{
```

```
return player;
```

```
}
```

```
@Override
```

```
public void update() {
```

```
    TextBox.update();
```

```
    eye.update();
```

```
    moveKeys.update();
```

```
    upKeys.update();
```

```
checkCloseToBorderX();
```

```
checkCloseToBorderY();
```

```
//Se não estiver pausado, o jogo está rodando
```

```
if(!paused)
```

```
{
```

```
levelManager.update();
```

```
}
```

```
triggerskychange.update();
```

```
pauseOverlay.update();
```

```
updateAllThings();
```

```
}
```

```
@Override
```

```
public void draw(Graphics g)
```

```
{  
  
if(paused)  
  
{  
  
pauseOverlay.pauseTrigger=true;  
  
}  
  
backGroundParallax(g);  
  
levelManager.draw(g, xLevelOffset, yLevelOffset);  
  
drawProps(g);  
  
drawTrash(g);  
  
drawTrampoline(g);  
  
drawCheckpoint(g);  
  
drawSpike(g);  
  
Props.draw(g, trashBig, 1631, 2624, xLevelOffset, yLevelOffset);  
  
Props.draw(g, trashBig, 2240-40, 2080, xLevelOffset, yLevelOffset);
```

```
drawOrbdash(g);
```

```
eye.draw(g, xLevelOffset, yLevelOffset, player, miniplayer);
```

```
moveKeys.draw(g, xLevelOffset, yLevelOffset, player, miniplayer);
```

```
upKeys.draw(g, xLevelOffset, yLevelOffset, player, miniplayer);
```

```
player.render(g);
```

```
garbageTruck.draw(g, 0, yLevelOffset);
```

```
drawTriggerEvent(g);
```

```
if (triggereventTwo.getTick() <= 13)
```

```
    TextBox.resetTick();
```

```
else if (triggereventTwo.getTick() > 13 && triggereventTwo.getCollisionTrigger()  
    !=2)
```

```
    TextBox.draw(g);
```

```
pauseOverlay.draw(g);
```

```
g.setColor(new Color(0,0,0,black));

g.fillRect(0, 0, Game.GAME_WIDTH, Game.GAME_HEIGHT);

}
```

```
public static void increaseBlackScreen(int blackValue)

{

if (black<=255 - blackValue)

black+=blackValue;

else

black=255;

}
```

```
public static void decreaseBlackScreen(int blackValue)

{

if (black>=0 + blackValue)

black-=blackValue;

else

black=0;

}
```

```
private void checkCloseToBorderX()
```



```
{  
  
int playerX = (int)(player.getHitbox().x);  
  
int diff = playerX - xLevelOffset;  
  
if(diff > rightBorder)  
{  
  
xLevelOffset += diff - rightBorder;  
  
}  
  
else if(diff < leftBorder)  
{  
  
xLevelOffset += diff - leftBorder;  
  
}  
  
if(xLevelOffset > maxLevelOffsetX)  
{  
  
xLevelOffset = maxLevelOffsetX;  
  
}  
  
else if(xLevelOffset < 0)  
{  
  
xLevelOffset = 0;
```

```
}
```

```
}
```

```
private void checkCloseToBorderY()
```

```
{
```

```
int playerY = (int)(player.getHitbox().y);
```

```
int diff = playerY - yLevelOffset;
```

```
if(diff > bottomBorder)
```

```
{
```

```
yLevelOffset += diff - bottomBorder;
```

```
}
```

```
else if(diff < topBorder)
```

```
{
```

```
yLevelOffset += diff - topBorder;
```

```
}
```

```
if(yLevelOffset > maxLevelOffsetY)
```

```
{
```

```
yLevelOffset = maxLevelOffsetY;
```

```
}  
  
else if(yLevelOffset < 0)  
  
{  
  
yLevelOffset = 0;  
  
}  
  
}  
  
}
```

Classe Entity:

Esta classe é interessante ser comentada, pois se trata de uma classe pai, em que diversas outras classes herdaram os atributos e métodos dela, serão consideradas entidades dentro jogo, centralmente, se resumem em objetos que contém uma colisão, como o jogador, os sacos de lixo, os “checkpoints” ou espinhos.

```
package entities;
```

```
import java.awt.Color;
```

```
import java.awt.Graphics;
```

```
import java.awt.Rectangle;
```

```
import java.awt.geom.Rectangle2D;
```

```
import gamestates.Playing;
```

```
import main.Game;
```

```
public abstract class Entity
```

```
{
```

```
protected float x, y;
```

```
protected int width, height;
```

```
//Rectangle para a hitbox de entidades, Player, sacos de lixo coletaveis ou espinhos
```

```
protected Rectangle.Float hitbox;
```

```
public Entity(float x, float y, int width, int height)
```

```
{
```

```
this.x = x;
```

```
this.y = y;
```

```
this.width = width;
```

```
this.height = height;
```

```
}
```

```
//Ver as hitboxes em funcionamento
```

```
protected void drawHitbox(Graphics g, Color color, int xLevelOffset, int yLevelOffset)
```

```
{
```

```
if (Playing.debugDrawHitbox)

{

g.setColor(color);

g.fillRect((int) hitbox.x - xLevelOffset, (int) hitbox.y - yLevelOffset, (int) hitbox.width,
(int) hitbox.height);

}

}
```

```
protected void initHitbox(float x, float y, int width, int height)

{

hitbox = new Rectangle2D.Float(x, y, width, height);

}
```

```
public Rectangle2D.Float getHitbox()

{

return hitbox;

}
```

```
public void setHitbox(Rectangle.Float hitbox)

{

this.hitbox = hitbox;
```

```
}
```

```
public void setWidthandHeight(int width, int height)
```

```
{
```

```
this.width = (int) (width * Game.SCALE);
```

```
this.height = (int) (height * Game.SCALE);
```

```
}
```

```
}
```

Classe Player:

E a seguir, a classe com mais atributos e métodos de todos os códigos, pois o jogador contém métodos de troca de animação, movimentação, onde se encontram movimentação lateral, gravidade, pulo, impulso, colisão com paredes e diversos métodos “*get*” e “*set*”, mas aqui não será possível remover quase nada, pois todos atuam em conjunto e são importantes de terem neste documento.

```
package entities;
```

```
import static utilz.Constants.PlayerConstants.*;
```

```
import static utilz.HelpMethods.*;
```

```
import java.awt.Color;
```

```
import java.awt.Graphics;
```

```
import java.awt.image.BufferedImage;

import gamestates.Playing;

import main.Game;

import utiliz.LoadSave;


public class Player extends Entity

{

    //Matriz de localizacao dos sprites

    private BufferedImage[][] animations;


    //velocidade de animacao e qual sprite sera exibido na tela

    private int animationTick, animationIndex, animationSpeed = 30;


    private int[][] levelData;


    //Animações e velocidade de movimento, estado do player, estados dos
    botoes

    private int playerAction = IDLE;

    private boolean moving = false;

    private boolean up, left, down, right, jump, dashPressed;

    private float playerSpeed = 0.75f * Game.SCALE;
```

```
private float xSpeed = 0;
```

```
// Variaveis de pulo e gravidade
```

```
private float airSpeed = 0f;
```

```
private float wallDistance=3.5f * Game.SCALE;
```

```
private boolean gravityTrigger=false;
```

```
private float gravityValue = 0.06f;
```

```
private float gravity = gravityValue;
```

```
private float heavyGravity = 0.15f;
```

```
private float lightGravity = 0.01f;
```

```
private double wallJumpSpeed = 0.95;
```

```
private boolean jumpTrigger=false;
```

```
private static float jumpSpeedValue = -1.7f * Game.SCALE;
```

```
public static float jumpSpeed = jumpSpeedValue;
```

```
private float fallSpeedAfterCollision = 0.25f * Game.SCALE;
```

```
public boolean inAir = false;
```

```
public static boolean active = true;
```


//Variaveis do Dash

public static int dash = 0;

private int verticalDirection = 0;

private int horizontalDirection = 1;

private int dashMaxTime = 34, dashMinTime=10;

private float dashSpeed = 2.5f, divAirSpeedOne=3f, divAirSpeedTwo=1.25f;

public static boolean canDashAgain = true;

public boolean cantMove=false;

//Inverter o personagem para cada direcao

private int flipX = width;

private int flipW = -1;

private int flipAdd = -8;

//Configurações do quanto o sprite se desloca da Hitbox

private float xDrawOffset = 5.4f * Game.SCALE;

private float yDrawOffset = 7 * Game.SCALE;

public static int lastX = (int) (Playing.playerInitialX * Game.SCALE), lastY =
(int)(Playing.playerInitialY * Game.SCALE);

```
private static boolean restartTrigger=false;
```

```
private int tempX=0;
```

```
private int tempY=0;
```

```
public Player(float x, float y, int width, int height)
```

```
{
```

```
    super(x, y, width, height);
```

```
    loadAnimations();
```

```
    //Tamanho da hitbox do player
```

```
    initHitbox(x, y,(int) (6 * Game.SCALE), (int) (11 * Game.SCALE));
```

```
}
```

```
public static void restartGame()
```

```
{
```

```
    Playing.active=true;
```

```
    restartTrigger = true;
```

```
    OrbDash.staticOrbTrigger=true;
```

```
}
```

```
private void updateRestartGame()

{

    if (restartTrigger)

    {

        hitbox.x = lastX;

        hitbox.y = lastY;

        if (!IsEntityOnFloor(hitbox, levelData))

        {

            Game.attemptCount++;

            setInAir(true);

            restartTrigger=false;

        }

    }

}
```

```
private void updateAnimationTick()

{

    animationTick++;

}
```

```
if (animationTick >= animationSpeed)

{

animationTick = 0;

animationIndex++;


if (playerAction != CROUNCH && playerAction != FALLING)

{

if (animationIndex >= GetSpriteAmount(playerAction))

{

animationIndex = 0;

}

}

else

{

if (animationIndex >= GetSpriteAmount(playerAction))

{

animationIndex = GetSpriteAmount(playerAction)-1;

}

}

}
```

```
}
```

```
private void setAnimation()
```

```
{
```

```
int startAnimation = playerAction;
```

```
if (Playing.active==true)
```

```
{
```

```
if (moving && dash==0)
```

```
{
```

```
animationSpeed=10;
```

```
playerAction = RUNNING;
```

```
}
```

```
else if (down && dash==0)
```

```
{
```

```
animationSpeed=5;
```

```
playerAction = CROUNCH;
```

```
}
```

```
else if(dash==0)
```

```
{

animationSpeed=60;

playerAction = IDLE;

}


if (inAir)

{

if (airSpeed < 0 && (dash==0 || dash>dashMaxTime))

{

if (down)

setGravity(heavyGravity);

else

setGravity(gravityValue);

animationSpeed=30;

playerAction = JUMP;

}

else

{

if (!(CanMoveHere(hitbox.x + wallDistance, hitbox.y-10, hitbox.width, hitbox.height,
levelData))
```

```
|| (!CanMoveHere(hitbox.x - wallDistance, hitbox.y-10, hitbox.width, hitbox.height,
levelData)))
```

```
{
```

```
if ((CanMoveHere(hitbox.x + wallDistance, hitbox.y, hitbox.width, hitbox.height,
levelData)))
```

```
{
```

```
if (dash==0 || dash>dashMaxTime)
```

```
{
```

```
flipAdd=0;
```

```
flipX = 0;
```

```
flipW = 1;
```

```
}
```

```
}
```

```
if ((CanMoveHere(hitbox.x - wallDistance, hitbox.y, hitbox.width, hitbox.height,
levelData)))
```

```
{
```

```
if (dash==0 || dash>dashMaxTime)
```

```
{
```

```
flipAdd = -5;
```

```
flipX = width;
```

```
flipW = -1;
```

```
}
```

```
}
```

```
if (gravityTrigger==false)
```

```
setGravityTrigger(true);
```

```
if (dash==0 || dash>dashMaxTime)
```

```
playerAction = CLIMB;
```

```
animationSpeed=30;
```

```
if (down)
```

```
setGravity(heavyGravity);
```

```
else
```

```
setGravity(lightGravity);
```

```
}
```

```
else if (dash==0 || dash>dashMaxTime)
```

```
{
```

```
animationSpeed=15;
```



```
playerAction = FALLING;
```

```
setGravityTrigger(false);
```

```
if (down)
```

```
setGravity(heavyGravity);
```

```
else
```

```
setGravity(gravityValue);
```

```
}
```

```
}
```

```
}
```

```
}
```

```
else
```

```
{
```

```
playerAction = DEATH;
```

```
}
```

```
if (startAnimation != playerAction)
```

```
resetAnimationTick();
```

```
}
```

```
private void setGravityTrigger(boolean graviyTrigger)

{

this.gravityTrigger=graviyTrigger;

}

private void resetAnimationTick()

{

animationTick = 0;

animationIndex = 0;

}


private void loadAnimations()

{

BufferedImage image = LoadSave.GetSpriteAtlas(LoadSave.PLAYER_ATLAS);


//Matriz dos sprites

animations = new BufferedImage[8][12];

for (int j = 0; j < animations.length; j++)

for (int i = 0; i < animations[j].length; i++)

animations[j][i] = image.getSubimage(i * 19, j * 19, 19, 19);
```

```
}
```

```
public void update()
```

```
{
```

```
if(!Playing.paused)
```

```
{
```

```
updatePosition();
```

```
updateAnimationTick();
```

```
updateDash();
```

```
}
```

```
setAnimation();
```

```
updateRestartGame();
```

```
}
```

```
private void updateDash()
```

```
{
```

```
if (dash > 0)
```

```
dash++;
```

```
if (dash > dashMaxTime)
```

```
{  
  
    setInAir(true);  
  
    //Verificar se o player cai no chao pra dai resetar a variavel  
  
    if (IsEntityOnFloor(hitbox, levelData))  
  
    {  
  
        dash=0;  
  
        canDashAgain=true;  
  
    }  
  
}  
  
  
if (dash > dashMinTime && dash < dashMaxTime && (Playing.active==true))  
  
{  
  
    airSpeed=0;  
  
  
    //Direções unicas  
  
    if ((verticalDirection == 0) && horizontalDirection == 1)  
  
    {  
  
        playerAction = DASH;  
  
        xSpeed = (dashSpeed / divAirSpeedTwo) * Game.SCALE;
```

```
//Direita
```

```
flipAdd = -5;
```

```
flipX = width;
```

```
flipW = -1;
```

```
}
```

```
if ((verticalDirection == 0) && horizontalDirection == -1)
```

```
{
```

```
playerAction = DASH;
```

```
xSpeed = (-dashSpeed / divAirSpeedTwo) * Game.SCALE;
```

```
//Esquerda
```

```
flipAdd=0;
```

```
flipX = 0;
```

```
flipW = 1;
```

```
}
```

```
if ((verticalDirection == 1) && horizontalDirection == 0)
```

```
{
```

```
playerAction = JUMP;
```

```
airSpeed = (-dashSpeed/divAirSpeedOne) * Game.SCALE;
```

```
}
```

```
if ((verticalDirection == -1) && horizontalDirection == 0)
```

```
{
```

```
playerAction = CROUNCH;
```

```
airSpeed = (dashSpeed) * Game.SCALE;
```

```
}
```

```
//Diagonais
```

```
if ((verticalDirection == 1) && horizontalDirection == 1)
```

```
{
```

```
playerAction = DASH;
```

```
xSpeed = (dashSpeed / 2f) * Game.SCALE;
```

```
airSpeed = (-dashSpeed /divAirSpeedOne / 1.4f) * Game.SCALE;
```

```
//Direita
```

```
flipAdd = -5;
```

```
flipX = width;
```

```
flipW = -1;
```

```
}
```

```
if ((verticalDirection == 1) && horizontalDirection == -1)
```

```
{
```

```
playerAction = DASH;
```

```
xSpeed = (-dashSpeed / 2f) * Game.SCALE;
```

```
airSpeed = (-dashSpeed / divAirSpeedOne / 1.4f) * Game.SCALE;
```

```
//Esquerda
```

```
flipAdd=0;
```

```
flipX = 0;
```

```
flipW = 1;
```

```
}
```

```
if ((verticalDirection == -1) && horizontalDirection == 1)
```

```
{
```

```
playerAction = DASH;
```

```
xSpeed = (dashSpeed / 2f) * Game.SCALE;
```

```
airSpeed = (dashSpeed / 5) * Game.SCALE;
```

```
//Direita
```

```
flipAdd = -5;
```

```
flipX = width;
```

```
flipW = -1;
```

```
}
```

```
if ((verticalDirection == -1) && horizontalDirection == -1)
```

```
{
```

```
playerAction =DASH;
```

```
xSpeed = (-dashSpeed / 2f) * Game.SCALE;
```

```
airSpeed = (dashSpeed / 5) * Game.SCALE;
```

```
//Esquerda
```

```
flipAdd=0;
```

```
flipX = 0;
```

```
flipW = 1;
```

```
}
```

```
updateXPosition(xSpeed);
```



```
updateYPosition(airSpeed);
```

```
}
```

```
}
```

```
public void render(Graphics g)
```

```
{
```

```
if (active)
```

```
g.drawImage/animations[playerAction][animationIndex],
```

```
(int) ((hitbox.x - xDrawOffset + flipX + flipAdd) - Playing.xLevelOffset),
```

```
(int) (hitbox.y - yDrawOffset + 1) - Playing.yLevelOffset, width * flipW, height, null);
```

```
}
```

```
private void updatePosition()
```

```
{
```

```
if (dash < dashMinTime || dash > dashMaxTime)
```

```
{
```

```
moving = false;
```

```
if (!left && !right && !inAir && !down && !jump && !up)
```

```
{
```

```
return;
```

```
}
```

```
verifyDirection(up,left,down,right);
```

```
xSpeed = 0;
```

```
if (right && !left && !down && Playing.active==true)
```

```
{
```

```
xSpeed = playerSpeed;
```

```
}
```

```
if (left && !right && !down && Playing.active==true)
```

```
{
```

```
xSpeed = -playerSpeed;
```

```
}
```

```
if (jump && Playing.active==true)
```

```
{
```

```
jump();
```

```
}
```

```
if (Playing.active==true)
```

```
{
```

```
if ((CanMoveHere(hitbox.x + wallDistance, hitbox.y, hitbox.width, hitbox.height,  
levelData)))
```

```
{
```

```
if (left && !right && !down)
```

```
{
```

```
flipAdd=0;
```

```
flipX = 0;
```

```
flipW = 1;
```

```
}
```

```
}
```

```
if ((CanMoveHere(hitbox.x - wallDistance, hitbox.y, hitbox.width, hitbox.height,  
levelData)))
```

```
{
```

```
if (right && !left && !down)
```

```
{
```

```
flipAdd = -5;
```

```
flipX = width;
```

```
flipW = -1;
```

```
}
```

```
}
```

```
}
```

```
}
```

```
if (inAir)
```

```
{
```

```
if (CanMoveHere(hitbox.x, hitbox.y + airSpeed, hitbox.width, hitbox.height,  
levelData))
```

```
{
```

```
hitbox.y += airSpeed;
```

```
airSpeed += gravity;
```

```
if (dash < dashMinTime || dash > dashMaxTime)
```

```
updateXPosition(xSpeed);
```

```
}
```

```
else
```

```
{
```

```
//metodo para verificar se o player colidiu com o chao ou o teto
```

```
hitbox.y = GetEntityYPosUnderRoofOrAboveFloor(hitbox, airSpeed);
```

```
//Ao atingir o chao
```

```
if (airSpeed > 0)
```

```
{
```

```
    resetInAir();
```

```
    jumpSpeed=jumpSpeedValue;
```

```
    if (dash > dashMaxTime)
```

```
    {
```

```
        dash=0;
```

```
        canDashAgain=true;
```

```
    }
```

```
}
```

```
//Ao atingir o teto
```

```
else
```

```
    airSpeed = fallSpeedAfterCollision;
```

```
    if (dash < dashMinTime || dash > dashMaxTime)
```

```
        updateXPosition(xSpeed);
```

```
    }
```

```
}
```

```
else
```

```
{
```

```
if (dash < dashMinTime || dash > dashMaxTime)
```

```
updateXPosition(xSpeed);
```

```
if (!IsEntityOnFloor(hitbox, levelData))
```

```
setInAir(true);
```

```
}
```

```
if ((!right && left || !left && right) && !down &&
```

```
((CanMoveHere(hitbox.x + wallDistance, hitbox.y, hitbox.width, hitbox.height,  
levelData)) &&
```

```
(CanMoveHere(hitbox.x - wallDistance, hitbox.y, hitbox.width, hitbox.height,  
levelData))))
```

```
moving = true;
```

```
}
```

```
private void verifyDirection(boolean up, boolean left, boolean down, boolean right)
```

```
{
```

```
//Direções unicas
```

```
if (left && (!up && !down && !right))
```

```
{
```

```
horizontalDirection=-1;

verticalDirection=0;

}

else if (right && (!up && !down && !left))

{

horizontalDirection=1;

verticalDirection=0;

}

else if (up && (!left && !right && !down))

{

horizontalDirection=0;

verticalDirection=1;

}

else if (down && (!left && !right && !up))

{

horizontalDirection=0;

verticalDirection=-1;

}

//Diagonals
```

```
if (left && up && (!down && !right))
```

```
{
```

```
horizontalDirection=-1;
```

```
verticalDirection=1;
```

```
}
```

```
else if (left && down && (!up && !right))
```

```
{
```

```
horizontalDirection=-1;
```

```
verticalDirection=-1;
```

```
}
```

```
else if (right && up && (!down && !left))
```

```
{
```

```
horizontalDirection=1;
```

```
verticalDirection=1;
```

```
}
```

```
else if (right && down && (!up && !left))
```

```
{
```

```
horizontalDirection=1;
```

```
verticalDirection=-1;
```



```
}
```

```
}
```

```
//Player nao comecar voando, evitando quem ta jogando a ter que apertar algum  
botao para dai comecar o jogo de verdade
```

```
public void loadLevelData(int[][] levelData)
```

```
{
```

```
    this.levelData = levelData;
```

```
    if (!IsEntityOnFloor(hitbox, levelData))
```

```
        setInAir(true);
```

```
}
```

```
public void jump()
```

```
{
```

```
    if (jumpTrigger==false)
```

```
    {
```

```
        if (inAir)
```

```
        //Ele envia as coordenadas da hitbox, mas com um valor a mais para a direita e para  
        a esquerda, caso alguma
```

```
        //delas colidam com uma parede, se é possivel pular de novo
```

```
if ((CanMoveHere(hitbox.x + wallDistance, hitbox.y, hitbox.width, hitbox.height,
levelData)) &&
```

```
(CanMoveHere(hitbox.x - wallDistance, hitbox.y, hitbox.width, hitbox.height,
levelData)))
```

```
return;
```

```
else
```

```
{
```

```
//quanto mais pulos voce der na parede, a forca do pulo vai ficando mais fraca
```

```
jumpSpeed *= wallJumpSpeed;
```

```
}
```

```
setInAir(true);
```

```
airSpeed = jumpSpeed;
```

```
setJumpTrigger(true);
```

```
}
```

```
}
```

```
private void updateXPosition(float xSpeed)
```

```
{
```

```
if (CanMoveHere(hitbox.x + xSpeed, hitbox.y, hitbox.width, hitbox.height, levelData))
```

```
{  
  
    hitbox.x += xSpeed;  
  
}  
  
else  
  
{  
  
    hitbox.x = GetEntityXPositionNextToWall(hitbox, xSpeed);  
  
}  
  
}  
  
  
private void updateYPosition(float airSpeed)  
  
{  
  
    if (CanMoveHere(hitbox.x, hitbox.y + airSpeed, hitbox.width, hitbox.height,  
        levelData))  
  
    {  
  
        hitbox.y += airSpeed;  
  
    }  
  
    else  
  
    {  
  
        hitbox.y = GetEntityYPosUnderRoofOrAboveFloor(hitbox, airSpeed);  
  
    }  
  
}
```

//Getters and Setters do pulo, dash, gravidade

public void resetInAir()

{

setInAir(false);

airSpeed = 0;

}

public void setInAir(boolean inAir)

{

this.inAir = inAir;

}

public boolean getDashPressed()

{

return dashPressed;

}

public void setDashPressed(boolean dashPressed)

{

```
this.dashPressed = dashPressed;  
  
}
```

```
public int getDash()  
  
{  
  
return dash;  
  
}
```

```
public void setDash(int dash)  
  
{  
  
this.dash = dash;  
  
}
```

```
public float getJumpSpeed()  
  
{  
  
return jumpSpeed;  
  
}
```

```
public void setJumpTrigger(boolean jumpTrigger) {  
  
this.jumpTrigger = jumpTrigger;
```

```
}
```

```
public void setGravity(float gravity)
```

```
{
```

```
this.gravity=gravity * Game.SCALE;
```

```
}
```

```
//Getters and Setters dos botões do teclado e mouse
```

```
public boolean isUp()
```

```
{
```

```
return up;
```

```
}
```

```
public void setUp(boolean up)
```

```
{
```

```
this.up = up;
```

```
}
```

```
public boolean isLeft()
```

```
{
```

```
return left;
```

```
}
```

```
public void setLeft(boolean left)
```

```
{
```

```
    this.left = left;
```

```
}
```

```
public boolean isRight()
```

```
{
```

```
    return right;
```

```
}
```

```
public void setRight(boolean right)
```

```
{
```

```
    this.right = right;
```

```
}
```

```
public boolean isDown()
```

```
{
```

```
return down;
```

```
}
```

```
public void setDown(boolean down)
```

```
{
```

```
    this.down = down;
```

```
}
```

```
public void setJump(boolean jump)
```

```
{
```

```
    this.jump = jump;
```

```
}
```

```
public void setAirSpeed(float airSpeed)
```

```
{
```

```
    this.airSpeed = airSpeed;
```

```
}
```

```
public void setCantMove(boolean cantMove)
```

```
{
```



```
this.cantMove=cantMove;  
  
}
```

```
public boolean getCantMove()  
  
{  
  
return cantMove;  
  
}
```

```
public int getTempX() {  
  
return tempX;  
  
}
```

```
public int getTempY() {  
  
return tempY;  
  
}  
  
}
```

5. DESAFIOS E OBSTÁCULOS

5.1 Dificuldades enfrentadas

De antemão, um dos maiores desafios deste projeto como um todo, foi o de ser realizado apenas por uma pessoa, por onde teve que passar por todas as etapas de forma, mas mesmo tendo sido bem desafiador, foi bem divertido, principalmente na parte criativa de desenvolver mecânicas, arte visual e história.

Em relação ao relatório, não houve tantas dificuldades, visto que em semestres passados eu também os realizei sozinho, portanto, já tinha uma boa base, na parte do código, mais focado na parte de lógica, também não foi um grande problema, pois eu já realizei diversos projetos de jogos no passado, mas o com certeza a maior dificuldade foi em relação ao tempo, mesmo que eu já tenha prática em desenvolver jogos, eu jamais havia finalizado um, e ter de finalizar um em tão pouco tempo realmente foi um desafio extremo para mim e levarei os diversos conhecimentos aprendidos a projetos futuros, principalmente na questão de gestão de tempo.

E por mais incrível que pareça, mesmo com tanto conteúdo apresentado e com tantas coisas presentes no projeto, ele ainda não possui um nome, nenhum que fosse algo realmente interessante sem soar bobo, a única ideia de um título é baseado no nome de um outro jogo, chamado “*Super Chicken Jumper*”, que o usaria como um trocadilho, mudando para “*Super Gari Jumper*”, o que condiz com a proposta do jogo, um gari que escala montanhas e coleta lixo, mesmo não sendo um bom nome, será com ele que o projeto irá ser denominado.

5.2 Importância do trabalho

Foi de suma importância para a formação acadêmica, visão pessoal e profissional o trabalho aqui realizado, onde ensinamentos sobre o estudo de uma linguagem focada em orientação a objetos, relacionando-os de forma correta, a seguir com as boas práticas de programação de Java, técnicas de elaboração de códigos complexos, formatação de arquivos formais, edição de imagem e formulação de bons parágrafos serão levados a futuros trabalhos acadêmicos e para o mercado de trabalho.

O desenvolvimento do código, por sua vez, mostrou-se muito útil, visto que o que foi aprendido sobre lógica de programação com Java, mais tarde poderá ser entrelaçado com outras áreas que futuramente serão estudadas, tendo a experiência aprendida neste trabalho como uma forte base para aplicar nas diferenças e limitações de diversas linguagens que existem no mundo e nos milhares que ainda estão por vir no futuro.

Bibliografia

KAARIN, Kaarin Gaming, Platformer Tutorial - Java. Disponível em:<
https://www.youtube.com/watchv=6_N8QZ47toY&list=PL4rzdwwizLaxYmltJQRjq18a9gsSyEQQ-0>. Acesso em 24 de maio. 2024.

Getting random numbers in Java [duplicate]. Disponível em:<<https://stackoverflow.com/questions/5887709/getting-random-numbers-in-java>>. Acesso em 24 de maio. 2024.

Sites de ferramentas usados

DEMIRRAMON, Undertale Text Box Generator. Disponível em:<https://www.demirramon.com/generators/undertale_text_box_generator>. Acesso em 24 de maio. 2024.

EZGIF, GIF to Sprite Sheet converter. Disponível em:<<https://ezgif.com/gif-to-sprite>>. Acesso em 24 de maio. 2024.